AULA 13

Achar não é acertar

Etapa 7 · terça, 13/10/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

Achar não é acertar — e a média não conta quem ficou de fora.

Dois números de hoje: IoU 0,86 com 48 instâncias casadas de 120; e 100% de faixas detectadas na sombra, com 49 px de erro. Nenhum dos dois dá erro.

Cenas dos últimos episódios: onde paramos

- O slam_toolbox assumiu map -> odom, e a correção cresceu junto com a deriva
- O mapa foi salvo com o launch no ar, e medido: cobertura e concordância
- A figura mostrou ONDE o mapa errou, e não só quanto
- E navegação cedeu pela segunda vez — volta hoje, no fim, e cede de novo se precisar

Quatro peças voltam hoje, e a segunda é a que a aula cobra

- Aula 3 — a máscara HSV: continua certa, e hoje deixa de bastar
- Aula 9 — a régua declarada antes: hoje ela ganha uma coluna
- Aula 10 — o contrato do detector: três miolos, a mesma assinatura
- Aula 12 — medir contra a verdade: hoje a verdade é pixel a pixel

O que você sai sabendo fazer hoje

- Separar objetos que se encostam — e dizer quando não dá
- Medir instância com quatro números, sabendo o que cada um esconde
- Publicar a sua segmentação por instância em /vision/segmented
- Atribuir a falha de uma cadeia de visão à etapa certa, com evidência

A aula está ordenada por custo de perda

- 1. Da máscara à instância — vence 18/10
- 2. A régua que acusa a fusão — vence 18/10
- 3. Percepção veicular: cor, borda, reta — vence 20/10
- 4. O que a navegação acrescenta — vence 30/10, e é o que cede

O mapa do TP3 vence quinta, 15/10.

Antes da tarefa de hoje: o par mapa.pgm + mapa.yaml no repositório. Ele é pré-requisito do TP4 inteiro, e o comparador ficou na raiz do workspace de propósito — /tmp não sobrevive ao reboot.

PARTE 1

Da máscara à instância

A mesma máscara, duas leituras

```
cd "$PB_DIR/aula13-instancia"
python3 cena.py        # 12 quadros, 10 peças cada, 7 delas encostadas
python3 avaliar.py
```

A cena tem verdade pixel a pixel: cada instância carrega o seu id inteiro num PNG de rótulos. Sem polígono, sem RLE, conferível a olho.

O que a régua devolve

detector	verdades	previstas	casadas	erro cont.	IoU medio	fusoes
cor_conectada	120	72	48	40.0%	0.86	48
cor_watershed	120	120	120	0.0%	0.94	0

Determinístico: rodando de novo, sai idêntico. O que viaja entre máquinas é a razão entre os tempos, não o tempo.

A máscara era a mesma nos dois.

Mesmo cv2.inRange, mesma morfologia. O que mudou foi a LEITURA: connectedComponents responde "quantas manchas", e a pergunta era "quantos objetos". Onde duas peças se tocam há uma mancha e dois objetos.

Como o watershed separa o que se encosta

- 1. A transformada de distância vira relevo: longe da borda, mais alto
- 2. Dois objetos colados viram dois picos com um vale no meio
- 3. Os picos viram sementes; a água sobe a partir delas
- 4. A linha onde as frentes se encontram é o corte

O corte é uma fração da altura do relevo

aula13-instancia/instancia.py

```
dist = cv2.distanceTransform(m, cv2.DIST_L2, 5)
for cada componente da mascara:
    pico = dist[componente].max()
    sementes |= (dist > fracao * pico) & componente   # fracao = 0,60

marcadores = 1 (fundo) | 2.. (sementes) | 0 (a decidir)
cv2.watershed(imagem_suavizada, marcadores)
```

O corte é POR COMPONENTE, não global: senão a peça pequena é engolida pela vizinha maior.

O watershed precisa de gargalo. Sem gargalo, nenhum parâmetro resolve.

Dois retângulos encostados FACE CONTRA FACE não têm vale na transformada de distância. Caixas numa esteira, livros numa pilha: o caminho não é ajustar a fração, é mudar de evidência — profundidade, borda, ou modelo treinado.

PARTE 2

A régua que acusa a fusão

Quatro números, e o que cada um esconde

- Erro de contagem — não diz QUAIS, nem nada sobre a forma
- IoU médio dos casados — esconde TUDO sobre quem não foi encontrado
- Fusões — quantas verdades caíram numa previsão só
- Divisões — o caso espelho: uma verdade partida em duas

IoU médio 0,86. Casadas: 48 de 120.

O IoU médio só fala dos pares que casaram. Um detector que achasse UMA peça, perfeitamente, marcaria 1,00. Por isso esse número nunca viaja sozinho — e isso vale para qualquer média que você publique no relatório.

O maior IoU da tabela é do pior detector

detector	verdades	previstas	casadas	erro cont.	IoU medio	classe
cor_conectada	120	72	48	40.0%	0.86	100.0%
cor_watershed	120	120	120	0.0%	0.94	100.0%
modelo (YOLOv8-seg)	120	17	4	85.8%	0.89	0.0%

O modelo pré-treinado achou 17 instâncias onde havia 120 e acertou ZERO por cento das classes — ele conhece COCO, não o domínio de vocês. E marca IoU 0,89: o segundo melhor da tabela, com quatro casadas de cento e vinte. Custo: 148 ms por quadro.

A figura diz onde; o número diz quanto

```
cd "$PB_DIR/aula13-instancia"
python3 avaliar.py --imagem "$PB_DIR/docs/evidencias/tp3/instancia.png"
```

Verde onde casou, vermelho onde fundiu, âmbar onde dividiu — com a legenda embutida, para a figura ir ao relatório sozinha.

As linhas que ligam o segmentador ao SEU domínio

aula13_instancia/config/instancia.yaml

```
segmentador_instancia:
  ros__parameters:
    metodo: cor_conectada     # cor_conectada | cor_watershed
    area_min: 400.0           # abaixo disso e sal-e-pimenta, nao peca
    fracao_semente: 0.60      # o corte na transformada de distancia

    classes: [vermelho, azul, verde]
    faixa:
      vermelho: [0, 10, 170, 180, 120, 60]
```

Troque a lista de classes e as faixas pelas cores do seu domínio. Use o amostrar_hsv da Aula 3 para descobrir os números com a SUA iluminação.

O mesmo miolo, publicando no tópico que o TP3 cobra

```
ros2 launch aula13_instancia instancia.launch.py metodo:=cor_watershed

# no outro terminal
ros2 topic echo /vision/instancias --once      # a contagem
ros2 run rqt_image_view rqt_image_view         # escolha /vision/segmented
```

A imagem sai colorida POR INSTÂNCIA, não por classe: duas peças vermelhas vizinhas saem em cores diferentes. É assim que se vê que a separação aconteceu.

PARTE 3

Percepção veicular, sem instalar simulador

Três etapas, e cada uma falha de um jeito diferente

- Cor — máscara HSV da pintura, branca e amarela em faixas separadas
- Borda — Canny sobre o cinza, filtrado pela máscara e pela região de interesse
- Reta — Hough agrupando segmentos em faixa esquerda e direita, pelo sinal
- Saber QUAL etapa quebrou é a diferença entre consertar e chutar

Cinco condições, com a geometria conhecida

```
cd "$PB_DIR/aula13-veicular"
python3 cena_estrada.py
python3 avaliar_faixas.py
```

A verdade de cada quadro é o ângulo de cada faixa e o x onde ela cruza a base da imagem — escolhido para o erro ser julgável de olho.

A régua, e a segunda tabela que a explica

condicao	achou as duas	erro angular	erro na base	etapa que quebrou
limpo	100.0%	0.15 g	0.79 px	--
sombra	100.0%	5.78 g	49.55 px	RETA
apagada	0.0%	--	--	COR
noite	0.0%	--	--	BORDA
faixa_dupla	100.0%	0.15 g	0.81 px	nenhuma

O avaliador não diz só que piorou: ele conta pixels depois da cor, pixels depois do Canny e segmentos depois do Hough — e nomeia a etapa.

apagada e noite dão o mesmo 0% por motivos opostos.

Na apagada a máscara do lado esquerdo vai de 1.649 px para ZERO: morre na cor. Na noite a máscara passa inteira (3.260 px) e o Canny devolve 0 px: morre na borda. Com um número só, as duas são indistinguíveis — e você mexe no parâmetro errado.

A sombra marca 100% de detecção e erra 49 pixels.

O limite da sombra é uma reta longa e inclinada; o agrupamento é pelo SINAL da inclinação; então ela entra no grupo da esquerda estando do lado direito do quadro. O ajuste é correto — a entrada é que não é.

As quatro figuras que são a evidência do G3.5

```
cd "$PB_DIR/aula13-veicular"
mkdir -p "$PB_DIR/docs/evidencias/tp3/veicular"
python3 faixas.py --quadro estrada/sombra/quadro006.png \
  --etapas "$PB_DIR/docs/evidencias/tp3/veicular"
```

Saem 1-cor.png, 2-borda.png, 3-reta.png e 4-final.png. Elas mostram, em vez de afirmar, onde a cadeia se perdeu.

Não existe almoço grátis, e dá para medir o preço

- Canny mais baixo (10, 30) recupera a noite: 0% -> 100%
- E arruína a sombra: erro angular de 3,9 para 23,4 graus
- Segmentos espúrios: de 4 para 43 por quadro
- Não é ajuste melhor. É outro compromisso — diga qual você escolheu, e o que perdeu

PARTE 4

O que a navegação acrescenta

O piloto do mundo mínimo, e o que falta nele

- Não sabe onde está -> localização contra o mapa (o mapa é do TP3)
- Não sabe para onde vai -> um objetivo, enviado como ação (Aula 8)
- Não vê além do laser -> custos global e local, construídos do mapa
- Não sabe o que fazer quando trava -> comportamentos de recuperação

O objetivo do Nav2 é uma ação.

Com feedback periódico e cancelamento — exatamente as da Aula 8. Navegação não é assunto novo: é a montagem de peças que já estão no lugar. Vira entrega em 30/10.

Tarefa da semana: duas evidências para o TP3

- O QUE FAZER: instância do SEU domínio publicando e MEDIDA; veicular rodando
- O QUE USAR: cor_watershed, avaliar.py --imagem, o nó, faixas.py --etapas
- COMO ADEQUAR: as classes são suas, e a verdade também — dez quadros rotulados
- E diga, para a condição que falhou, QUAL etapa quebrou e como você sabe

Como saber que terminou

```
cd "$PB_DIR/aula13-instancia" && python3 avaliar.py     # com o SEU cenário
cd "$PB_WS" && source install/setup.bash
ros2 launch aula13_instancia instancia.launch.py &
sleep 10 && ros2 topic echo /vision/instancias --once
cd "$PB_DIR/aula13-veicular" && python3 avaliar_faixas.py
```

Guarde a tabela, a figura e as quatro etapas em docs/evidencias/tp3/. Quem rodou e não guardou fez o trabalho e não provou.

Desafio: o modelo pré-treinado vale no seu domínio?

- O avaliar.py tem um terceiro detector, com o MESMO contrato: YOLOv8-seg num venv
- Ele acha o que o watershed não acha — inclusive o encostado face a face?
- E acerta a CLASSE? Ele conhece COCO, não "peça vermelha do meu projeto"
- Qual dos três você entregaria, e o que perde na escolha? Resposta sem número é errada

Para a próxima aula, 20/10 — aterrissagem do TP3

- É o último encontro antes da entrega de sexta, 23/10
- Clínica dos gates abertos e fechamento das evidências
- Chegue com o mapa salvo, a instância publicando e as figuras do veicular
- A clínica é para o que não fechou — não para começar